CENTRO UNIVERSITÁRIO LUTERANO DE PALMAS Recredenciado pela Portaria Ministerial nº 1.162, de 13/10/16, D.O.U. nº 198, de 14/10/2016 AELBRA EDUCAÇÃO SUPERIOR - GRADUAÇÃO E PÓS-GRADUAÇÃO S.A.		**Avaliação de Grau 1** **Valor: 6.0**
Curso:	Disciplina: Computação em Nuvem	Data: 28/09/2026
Turma:	Professor: Fábio Castro	Nota:
Aluno:		

1. Contexto

O Setor de Atendimento ao Estudante do campus controla os objetos perdidos em uma planilha compartilhada. Quem encontra uma garrafa, um carregador ou um casaco entrega no balcão; quem perdeu precisa ir até lá e perguntar. Para resolver isso, um estagiário desenvolveu o **Achados & Perdidos**, uma pequena aplicação web que recebe requisições HTTP, consulta um banco PostgreSQL e guarda a foto de cada objeto encontrado.

O problema: a aplicação só funciona no notebook dele. O banco foi instalado manualmente, as fotos ficam em uma pasta qualquer do disco e as senhas estão anotadas em um papel. O estágio terminou, e a coordenação quer publicar o sistema na nuvem no próximo semestre.

Você foi contratado(a) como **engenheiro(a) de nuvem** para a primeira etapa: empacotar o sistema em contêineres, de forma que qualquer pessoa consiga subir o ambiente completo com um único comando, sem perder nenhum objeto cadastrado nem nenhuma foto quando os contêineres forem recriados. Além de funcionar, o ambiente precisa ser **explicado**: a próxima equipe vai ler a sua documentação para migrar tudo para a nuvem.

> **Missão**
>
> Entregar um repositório em que o comando `docker compose up -d --build` sobe a aplicação web e o banco de dados, com dados persistidos em volumes, e uma documentação que explique como e por que cada peça funciona.

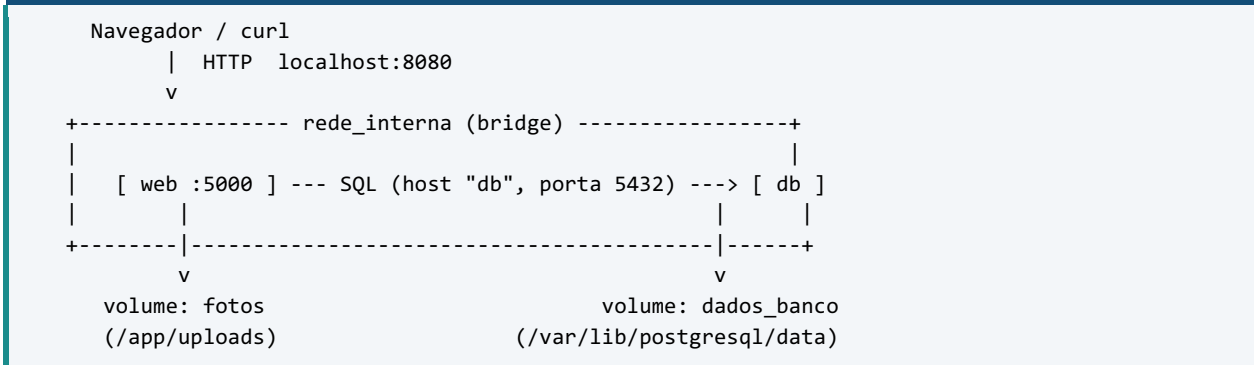
2. Arquitetura esperada

O ambiente é composto por dois serviços, uma rede privada e dois volumes nomeados. Apenas a aplicação web é acessível a partir da sua máquina (host); o banco de dados fica isolado na rede interna.

Componente	Tipo	Responsabilidade
`web`	Serviço (imagem própria)	API em Python/Flask servida pelo Gunicorn. Recebe as requisições HTTP na porta 5000 do contêiner, publicada como **8080** no host. Consulta o banco e grava as fotos no volume.
`db`	Serviço (imagem oficial `postgres:16-alpine`)	Banco PostgreSQL com a tabela `itens`. Criado automaticamente na primeira execução a partir do `init.sql`. **Não** deve ter porta publicada no host.
`rede_interna`	Rede bridge definida pelo usuário	Permite que `web` encontre `db` pelo nome (DNS interno do Docker).

Componente	Tipo	Responsabilidade
`dados_banco`	Volume nomeado	Diretório de dados do PostgreSQL (`/var/lib/postgresql/data`).
`fotos`	Volume nomeado	Fotos enviadas pelos usuários (`/app/uploads` no contêiner `web`).

Visão geral (esboço — o seu diagrama deve ser mais completo)



2.1 Requisitos

ID	Requisito
RF01	`GET /health` responde `"status": "ok"` e `"banco": "conectado"` quando o ambiente está no ar.
RF02	`GET /itens` lista os objetos cadastrados; `GET /itens?local=texto` filtra por local.
RF03	`POST /itens` cadastra um objeto (campos `descricao`, `local` e, opcionalmente, o arquivo `foto`).
RF04	`GET /fotos/<nome>` devolve a foto armazenada; `GET /estatisticas` mostra o total de itens no banco e de fotos no volume.
RNF01	Todo o ambiente sobe com `docker compose up -d --build`, sem nenhuma instalação manual além do Docker.
RNF02	Itens e fotos sobrevivem a `docker compose down` seguido de `docker compose up -d`.
RNF03	Nenhuma senha escrita no `Dockerfile`, no `docker-compose.yml` ou no código: credenciais vêm de um arquivo `.env` que **não** é versionado.
RNF04	O contêiner `web` roda com um usuário sem privilégios de root.
RNF05	O serviço `web` só inicia depois que o banco estiver pronto para aceitar conexões (não apenas "iniciado").

3. Recursos fornecidos

O código da aplicação **já está pronto** e não deve ser alterado: o foco do desafio é o ambiente, não a programação. Copie os três arquivos do Apêndice A para a estrutura de pastas da Tarefa 1.

Arquivo	Conteúdo
`app/app.py`	Aplicação Flask com as rotas dos requisitos RF01 a RF04. Lê a configuração das variáveis de ambiente `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` e `UPLOAD_DIR`.

Arquivo	Conteúdo
`app/requirements.txt`	Dependências Python: Flask, psycopg (driver PostgreSQL) e Gunicorn (servidor de produção).
`db/init.sql`	Cria a tabela `itens` e insere três objetos de exemplo.

> **Leia o código antes de começar**
>
> Descubra no `app.py` em qual diretório as fotos são gravadas, qual nome de host o código usa para o banco quando `DB_HOST` não é informado e em qual porta a aplicação deve escutar. Essas respostas determinam boa parte do seu `docker-compose.yml`.

4. Tarefas

TAREFA 1 — **Estrutura do repositório** — —

Organize o projeto exatamente assim (arquivos marcados com * são criados por você):

```
achados-perdidos/
|-- app/
|   |-- app.py              (fornecido)
|   |-- requirements.txt    (fornecido)
|   |-- Dockerfile          *
|   `-- .dockerignore       *
|-- db/
|   `-- init.sql            (fornecido)
|-- docs/                   *  diagrama, relatório e evidências
|-- docker-compose.yml      *
|-- .env                    *  NÃO entregar
|-- .env.example            *  modelo sem senha real
|-- .gitignore              *
`-- README.md               *
```

TAREFA 2 — **Dockerfile da aplicação web** — **1,5 pt**

Escreva o `app/Dockerfile` que gera a imagem do serviço `web`. Ele deve:

- partir de uma imagem oficial e enxuta do Python 3.12 (variante `slim`);
- copiar e instalar o `requirements.txt` **antes** de copiar o código, para aproveitar o cache de camadas;
- instalar as dependências sem guardar cache do pip na imagem;
- criar o diretório de uploads e um usuário sem privilégios, dono desse diretório, e executar a aplicação com esse usuário (RNF04);
- documentar a porta com `EXPOSE` e iniciar com o Gunicorn escutando em `0.0.0.0:5000` (forma *exec*, com colchetes).

  Crie também um `.dockerignore` que impeça `.env`, `__pycache__/` e arquivos `.pyc` de entrarem no contexto de build.

TAREFA 3 — Orquestração com Docker Compose — 2,5 pts

Escreva o `docker-compose.yml` que implementa a arquitetura da seção 2. Requisitos obrigatórios:

- serviço `db` com a imagem `postgres:16-alpine`, configurado pelas variáveis `POSTGRES_DB`, `POSTGRES_USER` e `POSTGRES_PASSWORD` lidas do `.env`;
- volume nomeado `dados_banco` montado no diretório de dados do PostgreSQL;
- o arquivo `db/init.sql` montado em `/docker-entrypoint-initdb.d/` como **somente leitura** (bind mount);
- `healthcheck` no `db` usando `pg_isready`;
- serviço `web` construído a partir de `./app`, com as variáveis de ambiente de que o `app.py` precisa, porta `8080:5000` e volume nomeado `fotos` montado no diretório de uploads;
- `depends_on` com `condition: service_healthy` (RNF05);
- rede definida pelo usuário `rede_interna` compartilhada pelos dois serviços;
- política `restart: unless-stopped` em ambos;
- nenhuma porta do `db` publicada no host e nenhuma senha escrita diretamente no arquivo (RNF03).

  O arquivo `.env.example` deve conter as mesmas chaves do `.env`, com valores fictícios. O `.gitignore` deve ignorar o `.env`.

TAREFA 4 — Subida do ambiente e testes funcionais — 1,5 pt

Suba o ambiente e execute os testes abaixo, guardando a saída de cada um (print ou texto) na pasta `docs/evidencias/`. No PowerShell do Windows, use `curl.exe` em vez de `curl`.

Roteiro de testes

```
docker compose up -d --build
docker compose ps                       # os dois serviços "running"; db "healthy"

curl http://localhost:8080/health
curl http://localhost:8080/itens
curl "http://localhost:8080/itens?local=biblio"

# cadastro com foto (use qualquer imagem .jpg ou .png sua)
curl -F descricao="Oculos de grau" -F local="Bloco B" -F foto=@oculos.jpg \
     http://localhost:8080/itens

curl http://localhost:8080/estatisticas
# abra no navegador a URL da foto retornada em GET /itens

docker compose logs web --tail 20
```

TAREFA 5 — Prova de persistência — 1,5 pt

Demonstre, com evidências, o comportamento dos volumes em três situações. Antes de cada passo, anote o resultado de `GET /estatisticas`.

1. **Recriação dos contêineres.** Cadastre pelo menos dois objetos com foto. Execute `docker compose down` e depois `docker compose up -d`. Os itens e as fotos continuam lá?

2. **Inspeção dos volumes.** Execute `docker volume ls` e `docker volume inspect` nos dois volumes. Onde os dados ficam fisicamente? Liste o conteúdo do volume de fotos com `docker compose exec web ls -l /app/uploads`.
3. **Remoção dos volumes.** Execute `docker compose down -v` e suba novamente. O que aconteceu com os itens cadastrados, com as fotos e com os três itens de exemplo do `init.sql`? Explique por quê.

TAREFA 6 — Artefatos explicativos — 3,0 pts

Produza os artefatos abaixo. Eles valem tanto quanto o ambiente funcionando: a próxima equipe precisa entender o que você fez sem ter de adivinhar.

Artefato	Conteúdo mínimo	Formato
A1 — README	Objetivo do projeto; pré-requisitos; passo a passo para subir, testar, parar e destruir o ambiente; tabela das rotas da API; como configurar o `.env` a partir do `.env.example`.	`README.md`
A2 — Diagrama de arquitetura	Host, portas publicadas, os dois contêineres, a rede, os dois volumes (com o caminho de montagem de cada um) e o bind mount do `init.sql`. Indique o que fica dentro e fora da rede interna.	`docs/arquitetura.png` ou Mermaid no Markdown
A3 — Relatório técnico	(a) explicação de **cada instrução** do Dockerfile e da ordem escolhida; (b) explicação de **cada bloco** do `docker-compose.yml`; (c) o caminho completo de uma requisição `POST /itens` com foto, do `curl` até o disco, em forma de lista numerada ou diagrama de sequência; (d) ciclo de vida dos volumes, com base na Tarefa 5.	`docs/RELATORIO.md`
A4 — Evidências	Saídas das Tarefas 4 e 5, além de `docker network inspect` da rede interna e `docker image ls` mostrando o tamanho da imagem gerada. Cada evidência com uma legenda de uma linha.	`docs/evidencias/`
A5 — Reflexão e equivalência em nuvem	Respostas às questões da seção 5 e a tabela de equivalência da seção 5.1.	`docs/REFLEXAO.md`

5. Questões de reflexão

Responda em 3 a 8 linhas cada, citando os seus próprios arquivos e evidências sempre que possível.

Q1. Por que o serviço `web` consegue se conectar ao banco usando o nome `db` em vez de um endereço IP? Quem resolve esse nome e o que mudaria se os contêineres fossem criados com `docker run` na rede bridge padrão?

Q2. Qual a diferença entre um **volume nomeado** e um **bind mount**? Justifique por que o `init.sql` foi montado como bind mount e os dados do banco como volume nomeado.

Q3. Se você alterar o `init.sql` (por exemplo, adicionando uma coluna) depois que o ambiente já subiu uma vez e executar `docker compose up -d` de novo, a mudança é aplicada? Por quê? Como aplicar a alteração sem perder dados?

Q4. Por que `depends_on` sozinho, sem `condition: service_healthy`, não garante que a aplicação consiga se conectar ao banco na primeira tentativa?

Q5. A ordem das instruções `COPY requirements.txt` → `RUN pip install` → `COPY app.py` influencia o tempo de build. Explique usando o conceito de cache de camadas e cite o que acontece quando só o `app.py` muda.

Q6. Por que a porta 5432 do banco não deve ser publicada no host? Considerando que as fotos podem mostrar documentos pessoais esquecidos (carteira de estudante, RG), relacione a sua configuração com a LGPD e proponha duas medidas adicionais de proteção.

Q7. Tente executar `docker compose up -d --scale web=3`. O que acontece e por quê? O que seria necessário para rodar três réplicas da aplicação? As réplicas poderiam compartilhar o volume `fotos` com segurança?

5.1 Do ambiente local para a nuvem

O ambiente local é um protótipo do que será publicado na nuvem. Preencha a tabela com o serviço gerenciado equivalente em cada provedor e escreva um parágrafo indicando qual peça do seu ambiente muda mais ao migrar e por quê.

Peça do ambiente local	Microsoft Azure	AWS	Google Cloud
Contêiner `web`			
Contêiner `db` + volume `dados_banco`			
Volume `fotos`			
`rede_interna`			
Arquivo `.env` (segredos)			
Imagem construída localmente			

6. Desafios bônus (até +1,0 pt)

- **Proxy reverso (+0,4).** Adicione um serviço `nginx` na frente do `web`, que passa a não publicar porta alguma. Só o `nginx` expõe a porta 8080. Atualize o diagrama.
- **Backup do volume (+0,3).** Documente e demonstre um backup do volume `fotos` para um arquivo `.tar.gz` usando um contêiner temporário (`docker run --rm` com o volume montado) e a restauração em um volume novo.
- **Painel do banco (+0,3).** Adicione o `adminer` em um *profile* do Compose (`profiles: ["debug"]`), de modo que ele só suba com `docker compose --profile debug up -d`.

7. Checklist de entrega

Entregue o link do repositório (ou um `.zip` **sem** o arquivo `.env`) no ambiente virtual até a data indicada.

- ☐ `docker compose up -d --build` funciona em uma pasta recém-clonada, após copiar `.env.example` para `.env`
- ☐ `GET /health` retorna `ok` e `conectado`
- ☐ Nenhuma senha no `Dockerfile`, no `docker-compose.yml` ou no repositório
- ☐ O banco não publica porta no host
- ☐ `docker compose exec web whoami` não retorna `root`
- ☐ Itens e fotos sobrevivem a `down` + `up`

- ☐ README com passo a passo testado do zero
- ☐ Diagrama com rede, volumes e portas
- ☐ Relatório técnico com as quatro partes (a) a (d)
- ☐ Evidências nomeadas e legendadas em `docs/evidencias/`
- ☐ Sete questões de reflexão respondidas e tabela de equivalência preenchida

> **Situações que zeram o critério correspondente**
>
> Senha real versionada no repositório (zera Docker Compose). Ambiente que só sobe com passos manuais não documentados (zera Funcionamento). Evidências que não correspondem ao código entregue (zera o critério em questão).

Apêndice A — Código fornecido

Copie os arquivos exatamente como estão. Não é necessário alterá-los para cumprir o desafio.

app/app.py

```
import os, uuid, socket
import psycopg
from flask import Flask, request, jsonify, send_from_directory

app = Flask(__name__)
UPLOAD_DIR = os.getenv("UPLOAD_DIR", "/app/uploads")
DB_URL = (f"host={os.getenv('DB_HOST', 'db')} port={os.getenv('DB_PORT', '5432')} "
          f"dbname={os.getenv('DB_NAME')} user={os.getenv('DB_USER')} "
          f"password={os.getenv('DB_PASSWORD')}")

def conectar():
    return psycopg.connect(DB_URL)

@app.get("/health")
def health():
    try:
        with conectar() as c:
            c.execute("SELECT 1")
        return jsonify(status="ok", banco="conectado", container=socket.gethostname())
    except Exception as e:
        return jsonify(status="erro", banco=str(e)), 503

@app.get("/itens")
def listar():
    local = request.args.get("local")
    sql = "SELECT id, descricao, local, foto, criado_em FROM itens"
    params = []
    if local:
        sql += " WHERE local ILIKE %s"
        params.append(f"%{local}%")
    with conectar() as c:
        linhas = c.execute(sql + " ORDER BY criado_em DESC", params).fetchall()
    return jsonify([
        {"id": l[0], "descricao": l[1], "local": l[2],
         "foto": f"/fotos/{l[3]}" if l[3] else None, "criado_em": l[4].isoformat()}
        for l in linhas])

@app.post("/itens")
def cadastrar():
    descricao = request.form.get("descricao")
    local = request.form.get("local")
    if not descricao or not local:
        return jsonify(erro="campos 'descricao' e 'local' sao obrigatorios"), 400
    nome_foto = None
    foto = request.files.get("foto")
    if foto and foto.filename:
        ext = os.path.splitext(foto.filename)[1].lower()
        nome_foto = f"{uuid.uuid4().hex}{ext}"
        foto.save(os.path.join(UPLOAD_DIR, nome_foto))
    with conectar() as c:
        novo_id = c.execute(
            "INSERT INTO itens (descricao, local, foto) VALUES (%s, %s, %s) RETURNING id",
            (descricao, local, nome_foto)).fetchone()[0]
    return jsonify(id=novo_id, foto=nome_foto), 201

@app.get("/fotos/<nome>")
def foto(nome):
    return send_from_directory(UPLOAD_DIR, nome)
```

```
@app.get("/estatisticas")
def estatisticas():
    with conectar() as c:
        total = c.execute("SELECT COUNT(*) FROM itens").fetchone()[0]
    arquivos = len(os.listdir(UPLOAD_DIR))
    return jsonify(itens_no_banco=total, fotos_no_volume=arquivos)
```

app/requirements.txt

```
flask==3.0.3
psycopg[binary]==3.2.3
gunicorn==23.0.0
```

db/init.sql

```
CREATE TABLE IF NOT EXISTS itens (
    id         SERIAL PRIMARY KEY,
    descricao  VARCHAR(200) NOT NULL,
    local      VARCHAR(100) NOT NULL,
    foto       VARCHAR(100),
    criado_em  TIMESTAMP NOT NULL DEFAULT NOW()
);
INSERT INTO itens (descricao, local) VALUES
    ('Garrafa termica azul', 'Biblioteca'),
    ('Carregador de notebook USB-C', 'Laboratorio 3'),
    ('Casaco preto com capuz', 'Cantina');
```